

INT531

Site Reliability Engineering

Capacity Planning, Performance และ Load Testing

*ป้ายนี้ Lab — ทดสอบโหลดบริการถอดเสียงจนหาจุดที่ SLO พัง

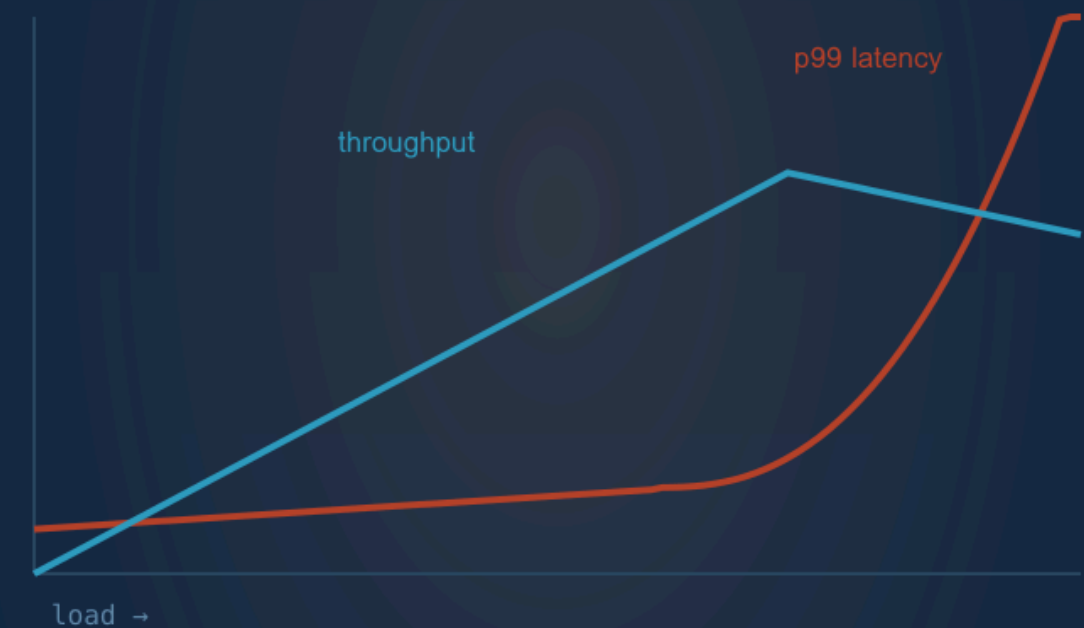
คณะเทคโนโลยีสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

บรรยาย 1.5 ชม. + ปฏิบัติการ 1.5 ชม. | ปรับปรุงเนื้อหา พ.ศ. 2569 (2026)

```
$ k6 run ramp.js
```

รับได้เท่าไร
ก่อนที่ SLO จะพัง

วัดให้รู้ ก่อนที่ผู้ใช้จะเป็นคนบอก



แผนการเรียนรู้วันนี้

เข้าใจว่ารับได้เท่าไร และป้องกันอย่างไรเมื่อเกิน · บ่าย ลงมือทดสอบจริง

01

Capacity Planning คืออะไร

การเติบโตสองแบบ และการแปลงความต้องการเป็นทรัพยากร

02

การทดสอบห้าแบบ

Load, Stress, Soak, Spike, Breakpoint และเลือกใช้ให้ตรงคำถาม

03

ออกแบบการทดสอบให้ผลเชื่อถือได้

Workload Model, Think Time และกัณฑ์ Coordinated Omission

04

อ่านผลและหาจุดที่ SLO พัง

Knee Point และ Maximum Sustainable Load

05

ป้องกันเมื่อโหลดเกิน

Rate Limiting, Load Shedding, Autoscaling และข้อจำกัด

06

ต้นทุนและ Lab

Cost per Request, Right-sizing และการทดสอบจริงภาคบ่าย

ผลลัพธ์การเรียนรู้ประจำคาบ

เมื่อจบคาบนี้ นักศึกษาจะสามารถ...

SLO 1	แยกแยะประเภทการทดสอบ และเลือกใช้ให้ตรงวัตถุประสงค์	CLO5
SLO 2	ออกแบบและรันการทดสอบโหลดพร้อมกำหนด Workload Model ที่สมจริง	CLO5
SLO 3	วิเคราะห์ผลเพื่อหา Knee Point และขีดความสามารถสูงสุดที่ยังคง SLO ไว้ได้	CLO5, CLO2
SLO 4	พยากรณ์ความต้องการทรัพยากรจากข้อมูลการเติบโต และคำนวณ Headroom	CLO5
SLO 5	อธิบายกลไกป้องกันการรับโหลดเกิน และข้อจำกัดของแต่ละกลไก	CLO5
SLO 6	ประเมินความคุ้มค่าด้านต้นทุนและผลด้านพลังงาน	CLO7

01

Capacity Planning คืออะไร

ตอบคำถามว่า ต้องมีเท่าไร ถึงจะยังรักษา SLO ไว้ได้ในวันที่คนใช้เยอะที่สุด

การเติบโตสองแบบ ที่ต้องวางแผนคนละวิธี

แบบหนึ่งค่อย ๆ มา อีกแบบมาทีเดียวและรู้วันล่วงหน้า

Organic — โตตามธรรมชาติ

- ผู้ใช้เพิ่มขึ้นเรื่อย ๆ ตามการใช้งานจริง
- พยากรณ์ได้จากข้อมูลย้อนหลัง
- วางแผนเป็นรอบ เช่น ทุกไตรมาส
- แก้ด้วยการเพิ่มทรัพยากรตามแผน

Inorganic — โตแบบก้าวกระโดด

- แคมเปญ ประกาศผล หรือวันลงทะเลเบียน
- พยากรณ์จากข้อมูลย้อนหลังไม่ได้ ต้องถามคนที่จัดงาน
- ต้องเตรียมก่อนวันงาน ไม่ใช่ตอนเกิดเหตุ
- แก้ด้วยการเตรียมความจุไว้ล่วงหน้าและกลไกป้องกัน

สัปดาห์ลงทะเลเบียนของคุณคือ Inorganic Growth ที่รู้วันล่วงหน้าเป็นเดือน • ไม่มีเหตุผลใดที่จะปล่อยให้ เป็นเซอร์ไพรส์

จากจำนวนผู้ใช้ ไปถึงจำนวนเครื่อง

ลำดับการแปลงที่ต้องทำให้ครบ ไม่ข้ามขั้น

1	เริ่มจากความต้องการทางธุรกิจ	คาดว่าจะมีผู้ใช้ 3,000 คนในสัปดาห์ลงทะเลเบียน	จำนวนผู้ใช้
2	แปลงเป็นอัตราคำขอ	คนละประมาณ 8 คำขอ กระจุกลงใน 2 ชั่วโมง	≈ 3.3 คำขอ/วินาที
3	คูณด้วยพีคที่ไม่เท่ากัน	ช่วงพีคจริงมักสูงกว่าค่าเฉลี่ย 3–5 เท่า	≈ 12–17 คำขอ/วินาที
4	เทียบกับขีดความสามารถที่วัดได้	จากการทดสอบโหลดจริง ไม่ใช่จากความรู้สึก	วัดได้ 20 คำขอ/วินาที
5	เผื่อกรณีเครื่องล้นและ Headroom	ต้องรันได้ด้วย N-1 และไม่ใช้เกิน 80%	ต้องมีความจุ ≈ 26

ให้คะแนนที่วิธีคิด ไม่ใช่ตัวเลขเป๊ะ · สิ่งที่ผิดไม่ได้คือลำดับขั้น การคูณพีค และการเผื่อ N-1 กับ 80%



การทดสอบห้แบบ

เลือกผิดแบบ คือการได้คำตอบของคำถามที่เราไม่ได้ถาม

ห้าแบบ ห้าคำถาม

และแบบที่ทีมส่วนใหญ่ลืมทำ

การทดสอบห้าแบบ — แต่ละแบบตอบคำถามคนละข้อ

Load	รับทราฟฟิกที่คาดหวังไว้ได้โดยยังอยู่ใน SLO หรือไม่ คงระดับที่คาดว่าจะฟื้นตัว 20–60 นาที
Stress	พังที่จุดไหน และพังแบบไหน ได้เกินพิกัดที่คาดว่าจะฟื้นตัว SLO จะพัง
Soak	อยู่ได้นานหลายชั่วโมงโดยไม่เสื่อมหรือไม่ คงระดับปกติไว้ 4–24 ชั่วโมง
Spike	ถ้าทราฟฟิกกระโดดในไม่กี่วินาทีจะเกิดอะไร กระโดดจากต่ำไปสูงมากภายในไม่ถึงหนึ่งนาที
Breakpoint	สูงสุดที่ระบบรับไหวอย่างต่อเนื่องคืออะไร ได้ขึ้นช้า ๆ จนละเมิด SLO ครั้งแรกแล้วหยุด

ทีมส่วนใหญ่รันแบบแรกแบบเดียวแล้วบอกว่าพิสูจน์ความจุแล้ว · แต่แบบที่สำคัญที่สุดสำหรับสัปดาห์ลงทะเลเบียนคือ Spike

03

ออกแบบการทดสอบให้ผลเชื่อถือได้

การทดสอบที่ออกแบบผิด ให้ตัวเลขที่ดูดีแต่ใช้ไม่ได้

ตัวอย่างที่ต้องกำหนดก่อนเขียนสคริปต์

ข้ามข้อใดข้อหนึ่ง แล้วผลที่ได้จะเถียงกันเองในที่ประชุม

Workload Model	สัดส่วนของแต่ละเส้นทางที่ผู้ใช้จริงทำ เช่น อ่าน 80% เขียน 20% · ยิ่งแต่ API ที่เบาที่สุดแล้วบอกว่ารับได้เยอะคือการหลอกตัวเอง
Think Time	เวลาที่ผู้ใช้จริงหยุดคิดระหว่างคลิก · ถ้าไม่ใส่ จะได้โหลดที่หนักกว่าความจริงหลายเท่าและได้ข้อสรุปที่ผิด
Baseline	ค่าที่วัดตอนโหลดต่ำก่อนเริ่มทดสอบจริง · ถ้าไม่มี จะแยกไม่ออกว่าระบบช้าลงหรือเครื่องที่ยิงทดสอบช้าเอง
เกณฑ์ผ่านคือ SLO	ตั้ง Threshold จาก SLO ที่ตกลงไว้ ไม่ใช่ตัวเลขที่รู้สึกว่าจะดี · ถ้ายังไม่มี SLO ให้กลับไปทำก่อน

ตัวอย่างสคริปต์ทดสอบ

k6 สำหรับบริการถอดเสียงของเรา · ใต้เป็นขั้น พร้อมเกณฑ์ที่มาจาก SLO

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  // ใต้เป็นขั้น ไม่ใช่กระโดด
  stages: [
    { duration: '2m', target: 50 },
    { duration: '2m', target: 150 },
    { duration: '2m', target: 300 },
    { duration: '2m', target: 450 },
    { duration: '1m', target: 0 },
  ],
  // เกณฑ์ผ่านคือ SLO ของเราเอง ไม่ใช่ค่าที่รู้สึกว่ายาก
  thresholds: {
    'http_req_duration': ['p(99)<1000'],
    'http_req_failed': ['rate<0.01'],
  },
};

export default function () {
  const res = http.post('https://stt.sit.kmutt.ac.th/api/transcribe',
    JSON.stringify({ lang: 'th' }),
    { headers: { 'Content-Type': 'application/json' } });
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(Math.random() * 3 + 1); // think time ของผู้ใช้งานจริง
}
```

stages

ใต้ทีละขั้นเพื่อให้เห็นว่าอาการเริ่มที่ระดับใด ไม่ใช่กระโดดไปสูงสุดทันที

thresholds

p99 ต่ำกว่า 1 วินาที และอัตราข้อผิดพลาดต่ำกว่า 1% คือ SLO ที่ตั้งไว้ตั้งแต่สัปดาห์ที่ 2

sleep

Think Time แบบสุ่ม เพื่อให้ไม่ทำให้ผู้ใช้งานลองทุกตัวyingพร้อมกันเป็นจังหวะเดียว

check

ตรวจว่าคำตอบถูกต้องจริง ไม่ใช่แค่ตอบเร็ว · ระบบที่ตอบ error เร็วมากก็ดูเหมือนเร็ว

กับดัก Coordinated Omission

สาเหตุที่ผลการทดสอบมักสวຍกว่าความจริงมาก

เกิดขึ้นได้อย่างไร

ผู้ใช้จำลองแบบปิดจะรอให้คำขอเดิมเสร็จก่อน จึงจะยิงคำขอถัดไป · เมื่อระบบช้าลง โหลดที่ป้อนเข้าไปจึงลดลงเอง

ทำไมจึงอันตราย

ช่วงที่ระบบช้าที่สุด กลับเป็นช่วงที่เราวัดน้อยที่สุด · ค่า p99 ที่ได้จึงต่ำกว่าความจริง

ผู้ใช้จริงไม่ทำแบบนั้น

ผู้ใช้จริงกดเข้ามาตามจังหวะของตัวเอง ไม่ได้รอให้คนก่อนหน้าเสร็จก่อน

วิธีแก้

ใช้แบบเปิดที่กำหนดอัตราการมาถึงของคำขอ เช่น ramping-arrival-rate ใน k6 หรือโหมดเทียบเท่าในเครื่องมืออื่น

คำที่ใช้เรียกปรากฏการณ์นี้มาจาก **Gil Tene** ผู้เขียน **HdrHistogram** · เป็นข้อผิดพลาดที่พบบ่อยที่สุดในการทดสอบโหลด

แบบปิดกับแบบเปิด ต่างกันอย่างไรในโค้ด

เปลี่ยนไม่กี่บรรทัด แต่เปลี่ยนความหมายของผลลัพธ์ทั้งหมด

```
// แบบปิด (closed model) – ค่าเริ่มต้นของ k6
// ผู้ใช้จำลองจะรอให้คำขอเดิมเสร็จก่อนจึงยิงคำขอถัดไป
// ระบบช้าลง = โหลดที่ป้อนเข้าไปลดลงเอง ← ช้อนปัญหาไว้

// แบบเปิด (open model) – ใกล้เคียงผู้ใช้จริงมากกว่า
export const options = {
  scenarios: {
    ramp: {
      executor: 'ramping-arrival-rate',
      startRate: 20, timeUnit: '1s',
      preAllocatedVUs: 100, maxVUs: 1000,
      stages: [
        { duration: '2m', target: 60 },
        { duration: '2m', target: 120 },
        { duration: '2m', target: 200 },
      ],
    },
  },
};
// คำขอมายังตามอัตราที่กำหนด ไม่ว่าระบบจะตอบทันหรือไม่
```

VUs

กำหนดจำนวนผู้ใช้จำลอง · ระบบช้าลงแล้วโหลดลดตาม

arrival rate

กำหนดอัตราคำขอต่อวินาที · ระบบช้าลงแล้วคิวยาวขึ้นเหมือนของจริง

preAllocatedVUs

เตรียมผู้ใช้จำลองไว้ล่วงหน้า เพื่อไม่ให้เกิดการสร้างเพิ่มกลายเป็นคอขวดเอง

ใช้แบบไหนดี

แบบเปิดสำหรับหาความจุ · แบบปิดสำหรับจำลองจำนวนผู้ใช้พร้อมกันที่แน่นอน

04

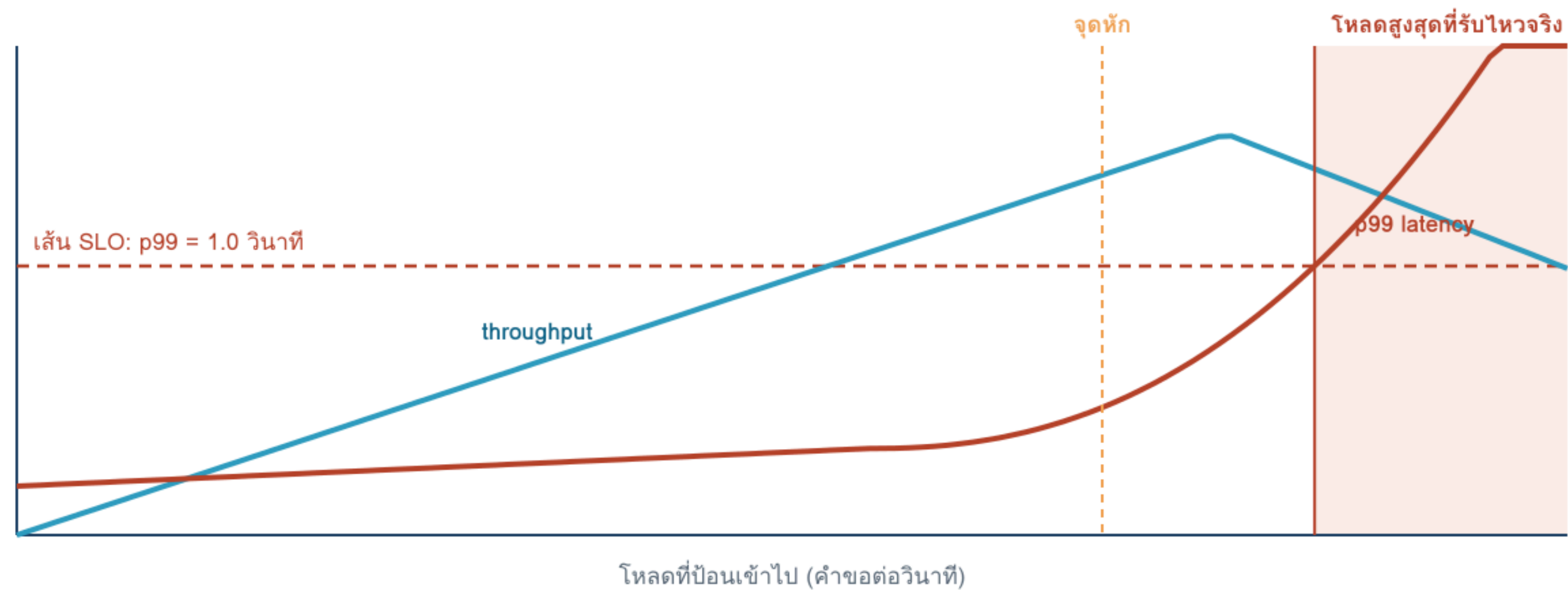
อ่านผลและหาจุดที่ SLO พัง

ตัวเลขเดียวที่ต้องได้จากการทดสอบ คือรับได้เท่าไรก่อนที่ SLO จะพัง

อ่านกราฟให้เป็น

Throughput กับ Latency เล่าคนละเรื่อง ต้องดูพร้อมกัน

อ่านผลการทดสอบ — หาจุดหักและจุดที่ SLO เริ่มพัง



โหลดสูงสุดที่รับไหว ไม่ใช่จุดที่ระบบล้ม แต่คือจุดสุดท้ายที่ SLO ยังผ่าน แล้วหักส่วนเผื่อออกอีก

อ่านผลจากเครื่องมือจริง

สรุปผลจาก k6 หลังรันแบบไต่ขั้น

```
k6 run --out json=result.json ramp.js

scenarios: (100.00%) 1 scenario, 400 max VUs, 11m30s max duration

http_req_duration.....: avg=612ms  p(95)=1.42s  p(99)=3.08s
  { expected_response:true }...: avg=486ms  p(95)=1.09s  p(99)=2.11s
http_req_failed.....: 2.14%  x 1289  ✓ 58934
http_reqs.....: 60223  87.2/s
iteration_duration.....: avg=4.31s
vus.....: 400  min=10 max=400

x p99 under 1s
  ↳ 0% - ✓ 0 / x 1
✓ error rate under 1%

# the threshold failed: this run already exceeded the SLO
# so the answer is somewhere below 87 requests per second
```

บรรทัดที่สำคัญที่สุดคือ **threshold** ที่ไม่ผ่าน · **p99 = 3.08** วินาที เกิน **SLO** ที่ **1** วินาที แปลว่าคำตอบอยู่ต่ำกว่า **87** คำขอต่อวินาที ต้องได้ละเอียดขึ้นเพื่อหาจุดที่แท้จริง

คำนวณโหลดสูงสุดที่ใช้งานได้จริง

จากตัวเลขที่วัดได้ ไม่ใช่จากที่หวังไว้

1	จุดที่ SLO พังครั้งแรก	จากการทดสอบได้ชั้น p99 เกิน 1 วินาทีที่ระดับนี้	80 คำขอ/วินาที
2	หักส่วนเผื่อความปลอดภัย	ไม่ใช่เกิน 80% ของจุดนั้น ตามเส้นโค้งจากสปีดอัพที่ 3	64 คำขอ/วินาที
3	เผื่อกรณีเครื่องล้มหนึ่งเครื่อง	มี 3 เครื่อง ต้องรันได้ด้วย 2 จึงหารด้วย 3 คูณ 2	ประมาณ 43 คำขอ/วินาที
4	เทียบกับความต้องการที่พยากรณ์ไว้	พีคที่คาดไว้คือ 17 คำขอ/วินาที	ผ่าน มีส่วนเผื่อประมาณ 2.5 เท่า

ถ้าตัวเลขสุดท้ายต่ำกว่าพีคที่พยากรณ์ไว้ มีสามทางเท่านั้น: เพิ่มความจุ ลดความต้องการ หรือยอมรับว่า SLO จะพังในวันนั้น · ทั้งสามทางต้องบันทึกเป็นการตัดสินใจ

05

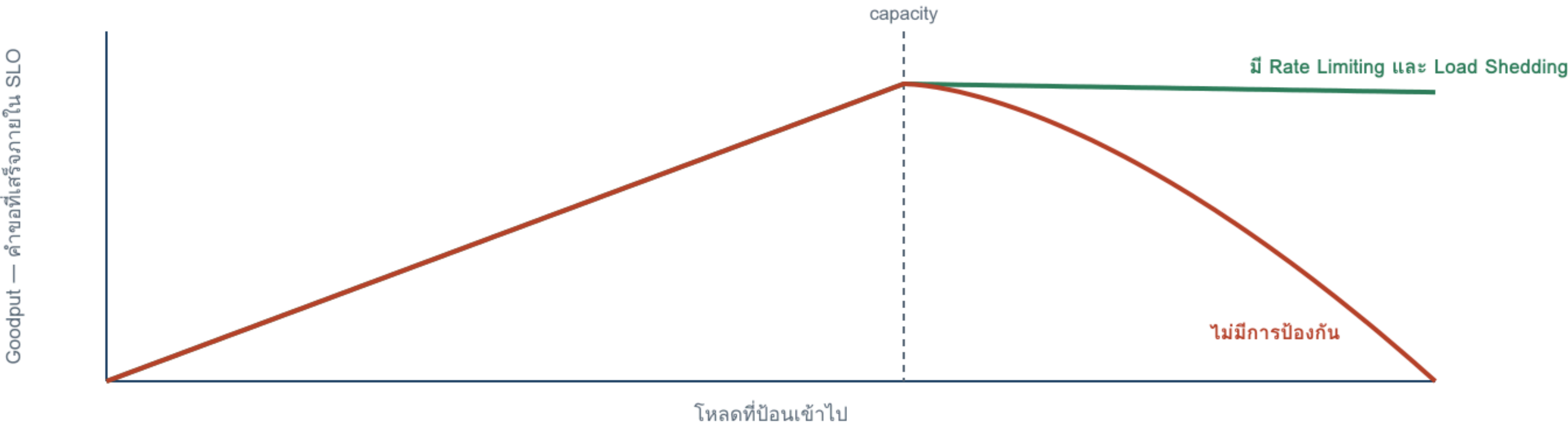
ป้องกันเมื่อโหลดเกิน

วันที่โหลดเกินความจุจะมาถึงเสมอ คำถามคือระบบจะแย่งอย่างไร

ยอมทิ้งงานบางส่วน เพื่อให้ที่เหลือยังใช้ได้

Goodput คือสิ่งที่ต้องรักษา ไม่ใช่ Throughput

การป้องกันโหลดเกิน — อะไรเปลี่ยนไปเมื่อยอมทิ้งงานบางส่วน



ทุกอย่างช้าลงพร้อมกัน คิวยาวขึ้น และสุดท้ายไม่มีใครได้รับบริการเลย

เมื่อเกินความจุ ส่วนเกินถูกปฏิเสธอย่างรวดเร็วและถูกทราฟฟิกที่รับไว้จึงยังอยู่ใน SLO

ปฏิเสธคำขอใน 5 มิลลิวินาที ดีกว่าตอบคำขอนั้นใน 30 วินาที เพราะผู้ใช้ลองใหม่ได้ และระบบยังไม่ล่ม

สึกลไกป้องกัน และข้อจำกัดของแต่ละอย่าง

ใช้ร่วมกัน ไม่ใช่เลือกอย่างใดอย่างหนึ่ง

กลไก	ทำงานอย่างไร	ข้อจำกัดที่ต้องรู้
Rate Limiting	จำกัดอัตราค่าขอต่อผู้ใช้หรือต่อ API ด้วยวิธีเช่น Token Bucket	ถ้าตั้งเพดานสูงเกินไปก็ไม่ได้ป้องกันอะไร · ต้องตั้งจากตัวเลขที่วัดได้
Load Shedding	ปฏิเสธค่าขอส่วนเกินอย่างรวดเร็วเมื่อคิวยาวเกินกำหนด	ต้องปฏิเสธให้ถูก โดยทิ้งงานที่สำคัญน้อยกว่าก่อน ไม่ใช่สุมทิ้ง
Graceful Degradation	ลดคุณภาพแทนที่จะหยุดให้บริการ เช่น ใช้โมเดลที่เล็กลงเมื่อคิวยาว	ต้องออกแบบไว้ล่วงหน้า และต้องบอกผู้ใช้ให้รู้ว่าอยู่ในโหมดลดคุณภาพ
Autoscaling	เพิ่มจำนวนหน่วยประมวลผลตามภาระที่วัดได้	ตอบสนองช้ากว่า Spike เสมอ เพราะต้องวัด ตัดสินใจ สร้าง และวอร์มอัป

ข้อควรระวังเรื่อง **Autoscaling**: ระหว่างที่หน่วยใหม่กำลังสร้าง ระบบยังอยู่ในสภาพโหลดเกิน · **Autoscaling** จึงลดต้นทุนได้ดี แต่รับ **Spike** ไม่ทันถ้าไม่มีส่วนเผื่อไว้ก่อน

06

ต้นทุนและปฏิบัติการ

ความจุที่มากเกินไปก็เป็นปัญหา เพียงแต่เป็นปัญหาคงแบบ

มองความจุในมุมมองต้นทุน

ตัวเลขที่ทำให้คุยกับฝ่ายบริหารได้

Cost per Request	เอาต้นทุนรวมต่อเดือนหารด้วยจำนวนคำขอ · ทำให้เทียบได้ว่าการปรับแต่งครั้งหนึ่งประหยัดได้เท่าไรจริง
Right-sizing	ปรับขนาดทรัพยากรให้ตรงกับที่วัดได้จริง · เครื่องที่ใหญ่เกินไปจำเป็นคือค่าใช้จ่ายที่ไม่ได้ซื้อความน่าเชื่อถือเพิ่ม
ส่วนเผื่อไม่ใช่ความสิ้นเปลือง	ส่วนเผื่อคือสิ่งที่ซื้อเวลาให้ทีมในวันที่โหลดฟุ้ง · แต่ต้องเป็นตัวเลขที่อธิบายได้ ไม่ใช่เผื่อไว้เฉย ๆ
พลังงานและความยั่งยืน	ในห้องของเราต้นทุนคือไฟฟ้าและความร้อน · เครื่องที่เปิดทิ้งไว้โดยไม่มีใครใช้ กินทั้งไฟและความจุของ PDU

เชื่อมกับสไลด์ที่ 4: ตัวเลขกำลังไฟที่คำนวณไว้ในไฟล์ BOM คือต้นทุนความจุในรูปแบบที่จับต้องได้ที่สุดของห้องเรา

ลำดับงานของแล็บ

เจ็ดขั้น ใช้ชุด int531-stt-monitoring ที่รันอยู่แล้วเป็นเป้าหมาย



ถ้าไม่มีค่าฐานจากขั้นที่ 1 จะแยกไม่ออกว่าระบบช้าลง หรือเครื่องที่ยิงทดสอบเองต่างหากที่ช้า

เกณฑ์ว่าแล็บผ่าน

หกข้อ ทุกข้อต้องมีตัวเลขและกราฟประกอบ

☐	มีค่า Baseline ที่วัดตอนโหลดต่ำ ก่อนเริ่มทดสอบจริง	ตารางค่าที่วัดได้
☐	สคริปต์มี Think Time และ Threshold ที่มาจาก SLO	ไฟล์สคริปต์ที่ใช้จริง
☐	กราฟ Throughput, p99 และอัตราข้อผิดพลาด ในการไต่ขั้นเดียวกัน	กราฟสามเส้นบนแกนเวลาเดียวกัน
☐	ระบุจุดที่ SLO พังครั้งแรก พร้อมค่าที่อ่านได้	ตัวเลขและตำแหน่งบนกราฟ
☐	คำนวณโหลดสูงสุดที่ใช้งานได้ พร้อมแสดงวิธีคิด	การคำนวณที่ละเอียด
☐	เปิด Rate Limiting แล้วทดสอบซ้ำ เทียบพฤติกรรม	กราฟทั้งสองรอบพร้อมคำอธิบาย

ให้คะแนนที่วิธีคิดและการอ่านผล ไม่ใช่ที่ตัวเลขสูง · กลุ่มที่รับได้ 20 ค่าขอต่อวินาทีแต่อธิบายได้ครบ ได้คะแนนมากกว่ากลุ่มที่อ้างว่ารับได้ 500 โดยอธิบายไม่ได้

การบ้านและสัปดาห์หน้า

หัวข้อถัดไปตามลำดับที่ปรับใหม่

งานกลุ่ม

- ผลแล็บครบหกข้อพร้อมกราฟ
- ระบุโหนดสูงสุดที่ใช้งานได้ของบริการกลุ่มตัวเอง
- เปรียบกับพีคที่พยากรณ์ไว้ แล้วสรุปว่าพอหรือไม่พอ

งานเดี่ยว

- เลือกบริการหนึ่งอย่างที่ใช้จริงในคณะ
- ประมาณพีคของมันด้วยวิธีคิดในสไลด์ที่ 6
- ระบุสมมติฐานทุกข้อที่ใช้ และสิ่งที่ยังไม่รู้

เตรียมล่วงหน้า

- ทบทวนเส้นโค้ง Utilization กับ Latency จากสัปดาห์ที่ 3
- อ่าน Google SRE Book บทที่ 21 (Handling Overload)
- เตรียมคำถามเรื่อง Autoscaling มาถามในคาบหน้า

สรุปสามประโยคของวันนี้

01

ความจุที่ไม่ได้วัด คือความจุที่เราเดาเอา · ตัวเลขเดียวที่ต้องได้จากการทดสอบคือรับได้เท่าไรก่อนที่ SLO จะพัง

02

การทดสอบที่ออกแบบผิดให้ตัวเลขที่ดูดีแต่ใช้ไม่ได้ · ถ้าไม่มี Think Time ไม่มี Baseline และไม่ใช้แบบเปิด ผลที่ได้จะสวยกว่าความจริงเสมอ

03

วันที่โหลดเกินความจุจะมาถึงเสมอ · สิ่งที่เราเลือกได้คือระบบจะปฏิเสธบางส่วนอย่างเป็นระเบียบ หรือล่มพร้อมกันทั้งหมดจนไม่มีใครได้รับบริการ

คาบถัดไป · **Kubernetes Reliability** — เมื่อความจุถูกจัดสรรและปรับขนาดโดยระบบ

เอกสารอ้างอิง

เรียงตามลำดับที่ควรอ่าน

ตำราและเอกสารหลัก

- Beyer, B. et al. (2016). Site Reliability Engineering. O'Reilly. บทที่ 21 (Handling Overload)
- Beyer, B. et al. (2018). The Site Reliability Workbook. O'Reilly. บทที่ 11 (Managing Load)
- เอกสารทางการของ k6 — หัวข้อ Scenarios, Executors และ Thresholds
- เอกสารทางการของ Locust สำหรับผู้ที่ถนัด Python

สำหรับอ่านต่อ

- Gil Tene — บรรยายเรื่อง Coordinated Omission และ HdrHistogram
- Gregg, B. (2020). Systems Performance (2nd ed.) — บทที่ 6 ด้วยวิธีการวัดและการทดสอบ
- เอกสารของรายวิชา — ชุด int531-stt-monitoring สำหรับใช้เป็นเป้าหมายการทดสอบ

หมายเหตุสำหรับผู้สอน

ไม่ต้องฉายในชั้นเรียน

ภาพในเด็กนี้

กราฟทั้งหมดเป็นภาพที่วาดขึ้นเพื่อการสอน แสดงรูปร่างของความสัมพันธ์ ไม่ใช่ข้อมูลที่วัดจากระบบจริง • หน้าจอ k6 เป็นผลลัพธ์จำลองที่จัดรูปแบบให้อ่านง่าย

ตัวเลขในตัวอย่างการคำนวณ

ทุกตัวเลขเป็นค่าสมมติเพื่อแสดงวิธีคิด • ให้นักศึกษาแทนค่าจริงของกลุ่มตัวเองเสมอ และให้คะแนนที่วิธีคิดตามนโยบายเดียวกับข้อสอบ

ข้อควรระวังด้านจริยธรรม

ต้องย้ำทุกครั้งว่าห้ามยิงทดสอบโหลดไปยังระบบที่ไม่ได้เป็นของตนเอง เพราะเข้าข่ายการโจมตี • ให้ยิงเฉพาะบริการของกลุ่มในสภาพแวดล้อมที่จัดไว้